01_CODE_ALONG_LAB

Pintop | 2026 Training

CLASSROOM CODE-ALONG - Routing, Middleware & Request Flow (Backend Design & Engineering)

Pintop Technologies | Class 03 | Instructor and student companion

What we will produce

Recreate a documented excerpt from the **real project** (`bootstrap/app.php`), practise a small safe variation, verify using the complete app or appropriate tool, and record both success and failure evidence. Preserve the original project.

Why: Requests arrive, and before your code even runs, several small checks should happen: is the user logged in, is the body readable, should this be logged. Today we see how a request flows through that chain so nothing is repeated in every handler.

1. Find and read the real file

`codebase/bootstrap/app.php`

The excerpt is in `REFERENCE EXCERPT.php`. Its job is to give you something to type, trace and understand. It is not guaranteed to run on its own: it is part of a larger file. Do not overwrite project source with a fragment.

2. Set up the real project

```
cd "Backend Design & Engineering/codebase"
composer install
cp .env.example .env
php artisan key:generate
php artisan migrate --seed
php artisan serve --port=8000
```

3. Type in stages

Stage 1

Say: "I am writing a small section of the actual project. Look for the inputs and names; we will connect it with the next block."

```
return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(web: __DIR__.'/../routes/web.php', api: __DIR__.'/../routes/api.php', apiPrefix: '', comm
    ->withMiddleware(function (Middleware $middleware): void {
```

Stage 2

Say: "I am writing a small section of the actual project. Look for the inputs and names; we will connect it with the next block."

```
        $middleware->alias(['training.token' => BearerToken::class]);
    })
    ->withExceptions(function (Exceptions $exceptions): void {
```

Stage 3

Say: "I am writing a small section of the actual project. Look for the inputs and names; we will connect it with the next block."

```
            // Keep the framework's default exception handling and validation responses.
        })->create();
```

4. Complete these activities

1. **Trace the request from route declaration through controller or handler to response.**
2. **Add a small local test request or safe demonstration route in a disposable branch; predict status and JSON.**
3. **Send a success request and a deliberately invalid request and explain both HTTP results.**

5. Check, debug and record

```
    php artisan route:list --path=api
    php artisan test
```

Expected check: If the local app starts or its tests pass, capture evidence of the specific behaviour tied to this class, not merely an application startup. Test one failed or edge input too. Log the exact error if blocked.

6. Marking

- 4/20: explain concept and input/output in plain language
- 4/20: accurate typed or edited work
- 4/20: local proof and explanation
- 4/20: safe variation and wrong-case recovery
- 4/20: short documented submission

Lab ethics: no unapproved public scanning, no third-party tokens and no real banking customer data.